Análisis general del código

```python
main.py > ...
def rana(n, k):
    maneras = [0] * (n + 1)
    maneras[0] = 1

    for i in range(1, n + 1):
        for salto in range(1, k + 1):
            if i - salto >= 0:
                maneras[i] += maneras[i - salto]

    return maneras[n]
def main ():
    n = int(input('Digita el numero de pasos:'))
    k = int(input('Digita el numero de saltos:'))
    maneras = rana(n, k)
    print(maneras)
main()
```

El código se encarda de almacenar dos variables, donde se le suma un n+1 cada ese n se recorre en un for donde va de 1 a n+1, donde ponemos unas condiciones para que el salto sea mayor o igual a cero, retorna el n o la posición, por así decirlo es un Fibonacci, pero guardando la posición en la que está.

Requerimientos:

H001. Entrada:

- Validar la posición de la rana
- Que sea un numero entero.

H002 -validar entrada

- Que sea entero
- Que valla de 1 hasta n+1
- Salto lo mismo
- Que sea mayor o igual a cero

H003-salidad

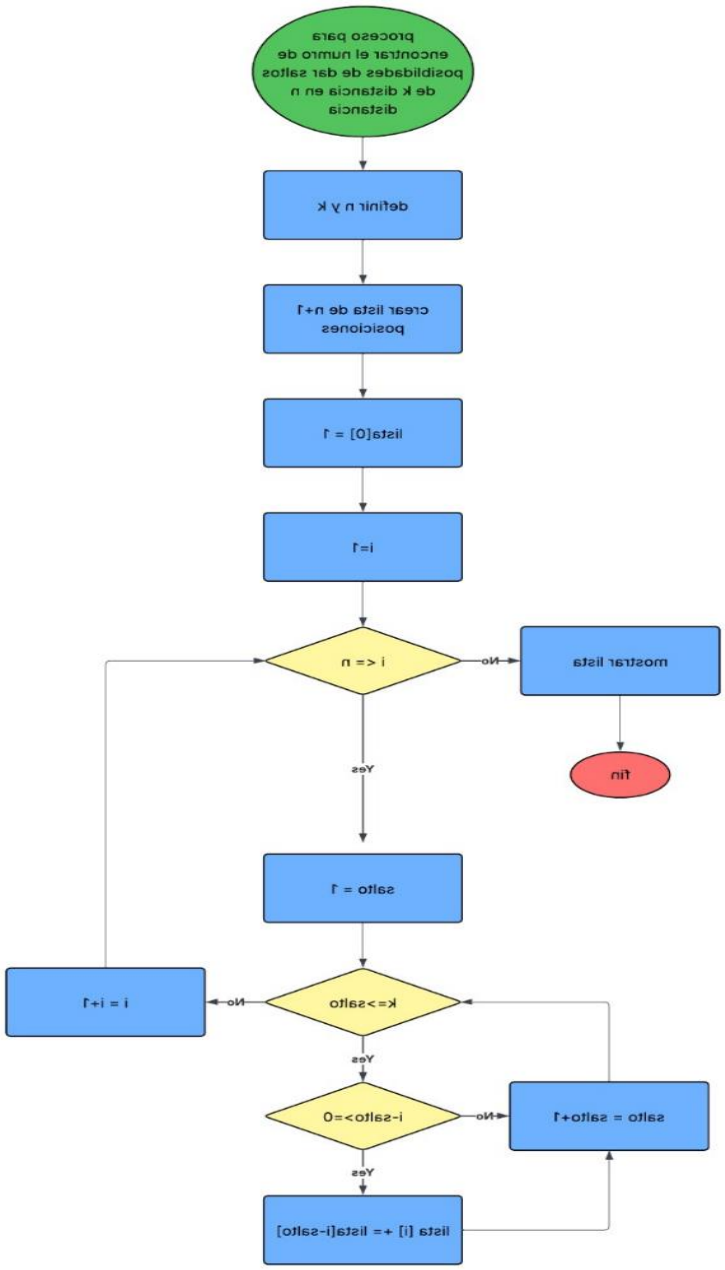
- Posición del sapo
- Saltos del sapo
- Maneras que puede recorrer.

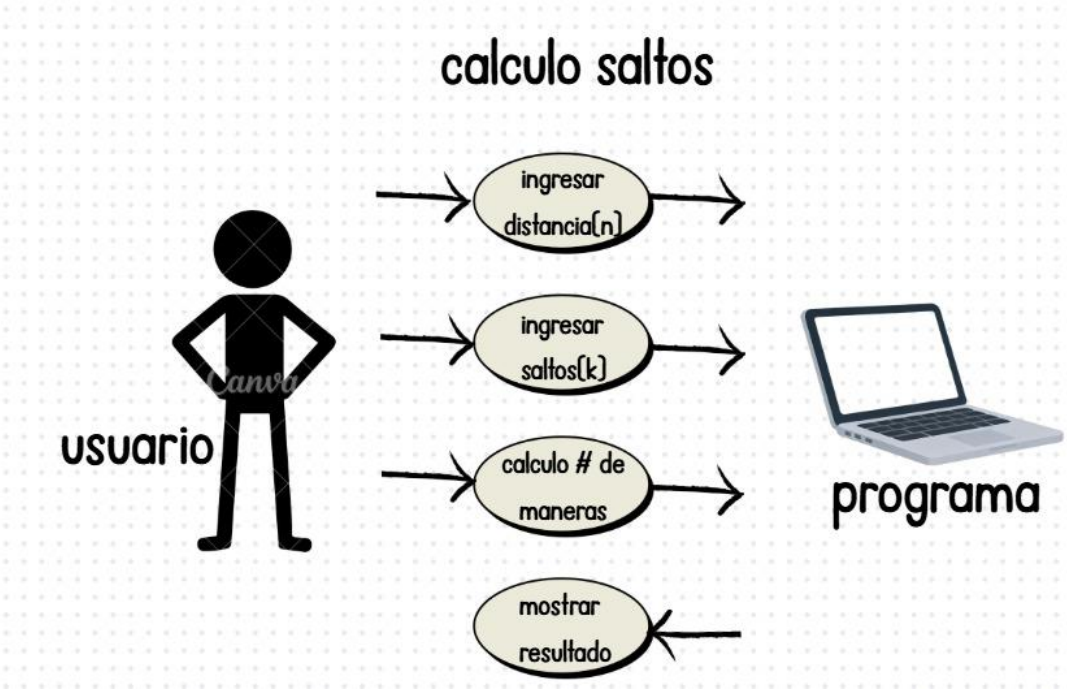
Tabla de complejidad:

COST	TIMES
C1	1
C2	1
C3	$n+1$
C4	n
C5+T(i-k)	1
C6	1
C7	1
C8	1
C9	1

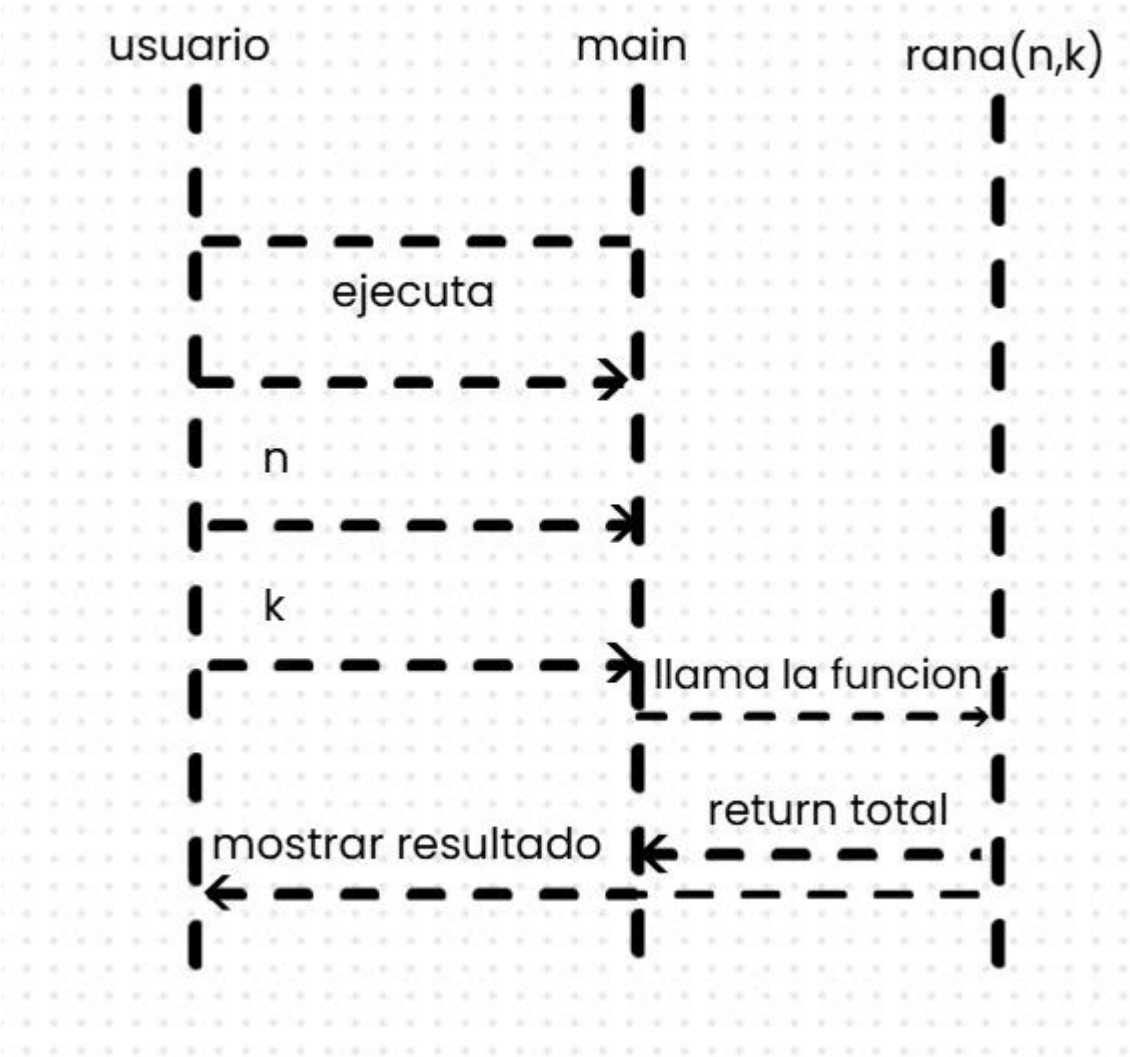
$$c1 + c2 + c3 + c4 + \big(c5 + (i - k)\big) + c6 + c7 + c8 + c9 = n^2$$

Diagrama de flujos:




calculo saltos
ingresar distancia(n)
ingresar saltos(k)
calculo # de maneras
mostrar resultado
usuario
programa

X|


usuario
main
rana(n,k)
ejecuta
n
k
llama la funcion
return total
mostrar resultado